Case Study 1: University Scholarship Allocation

1. Make

Build a Python program that screens scholarship applicants against academic, attendance, and family-income rules, then ranks eligible applicants.

2. Problem Statement

Scholarship applications must be checked consistently. The system should apply aggregate marks, minimum marks in each subject, attendance, and income criteria, then present eligible students in rank order.

3. Objective

Automate eligibility checking, store eligible records, rank applicants, and display a clear scholarship ranking list.

4. Assumptions

• Aggregate marks and subject marks are entered as numeric values.

• Eligibility requires aggregate marks ≥ 75, subject 1 ≥ 40, subject 2 ≥ 40, attendance ≥ 75, and family income ≤ ■250,000, matching the supplied code.

• Higher aggregate marks rank first; attendance and income are used as subsequent sort keys as implemented in the screenshot.

• Input values are assumed to be valid numbers.

5. Algorithm

1. Read the number of applicants.

2. For each applicant, read name, aggregate marks, both subject marks, attendance, and family income.

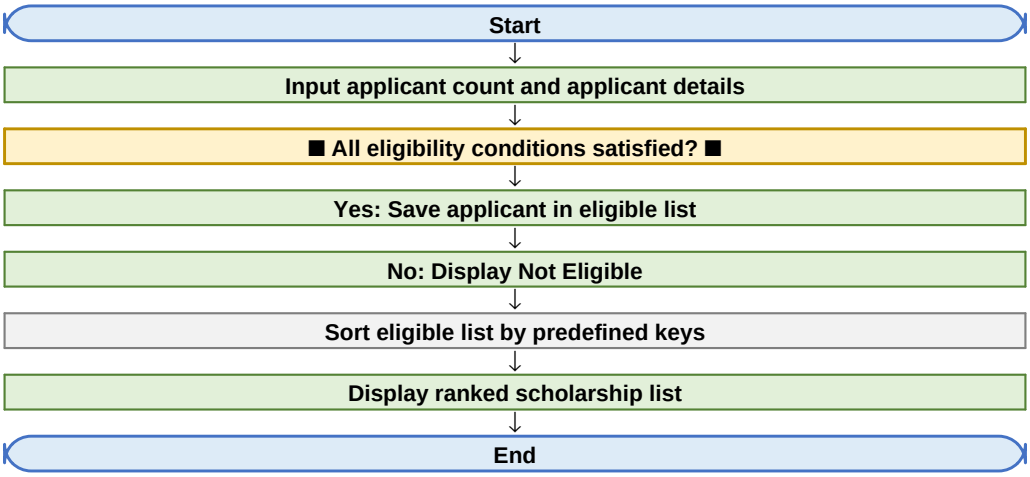
3. Check all five eligibility conditions.

4. If all conditions pass, save the applicant record and report Eligible; otherwise report Not Eligible.

5. Sort eligible records by aggregate marks, attendance, and income in descending tuple order, as coded.

6. Print the scholarship rank list.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

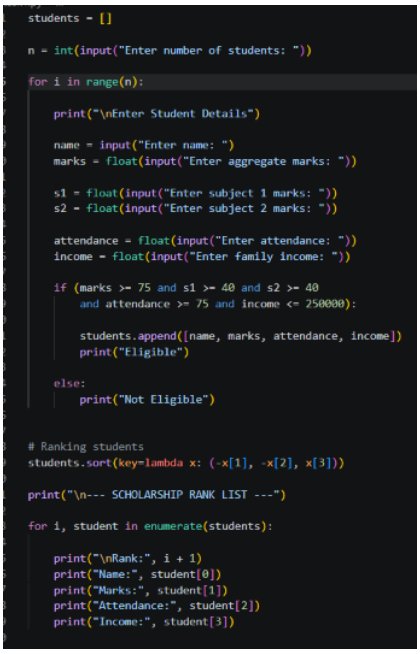
7. Pseudocode

```
BEGIN
READ&amp;nbsp;n
REPEAT&amp;nbsp;n&amp;nbsp;times:&amp;nbsp;READ&amp;nbsp;name,&amp;nbsp;marks,&amp;nbsp;s1,&amp;nbsp;s2,&amp;nbsp;attendance,&amp;nbsp;income
IF&amp;nbsp;marks&amp;nbsp;≥&amp;nbsp;75&amp;nbsp;AND&amp;nbsp;s1&amp;nbsp;≥&amp;nbsp;40&amp;nbsp;AND&amp;nbsp;s2&amp;nbsp;≥&amp;nbsp;40&amp;nbsp;AND&amp;nbsp;attendance&amp;nbsp;≥&amp;nbsp;75&amp;nbsp;AND&amp;nbsp;income&amp;nbsp;≤&amp;nbsp;250000&amp;nbsp;THEN
```

```
    APPEND applicant record to eligible list
ELSE DISPLAY 'Not Eligible'
SORT eligible list using ranking keys
DISPLAY ranked list
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for scholarship eligibility and ranking.



```python
students = []

n = int(input("Enter number of students: "))

for i in range(n):

    print("\nEnter Student Details")

    name = input("Enter name: ")
    marks = float(input("Enter aggregate marks: "))

    s1 = float(input("Enter subject 1 marks: "))
    s2 = float(input("Enter subject 2 marks: "))

    attendance = float(input("Enter attendance: "))
    income = float(input("Enter family income: "))

    if (marks >= 75 and s1 >= 40 and s2 >= 40
        and attendance >= 75 and income <= 250000):

        students.append([name, marks, attendance, income])
        print("Eligible")

    else:
        print("Not Eligible")


# Ranking students
students.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("\n--- SCHOLARSHIP RANK LIST ---")

for i, student in enumerate(students):

    print("\nRank:", i + 1)
    print("Name:", student[0])
    print("Marks:", student[1])
    print("Attendance:", student[2])
    print("Income:", student[3])
```

9. Conclusion

The program automates scholarship screening and produces a ranked list of qualified students. Clear thresholds make decisions consistent; the ranking keys can be adjusted to match the institution’s official tie-breaking policy.

Case Study 18: Fuel Consumption and Route Comparison

1. Make

Create a Python tool to compare fuel use and journey costs for multiple vehicles over a specified distance.

2. Problem Statement

A traveller needs to estimate fuel required and journey cost for different vehicles, including a reserve allowance, and identify the least-cost feasible vehicle.

3. Objective

Calculate fuel with reserve, estimate total journey cost, compare vehicles, and select the lowest-cost option.

4. Assumptions

• Distance is entered in kilometres, mileage in km/litre, and fuel price in currency per litre.

• Mileage must be greater than zero.

• Reserve is a percentage added to the calculated trip fuel.

• The supplied code compares recorded costs and selects the vehicle with the lowest cost; it does not explicitly test whether tank capacity is sufficient.

5. Algorithm

1. Read journey distance, fuel price, and reserve percentage.

2. Read the number of vehicles.

3. For each vehicle, read its name and mileage.

4. Reject non-positive mileage and continue to the next vehicle.

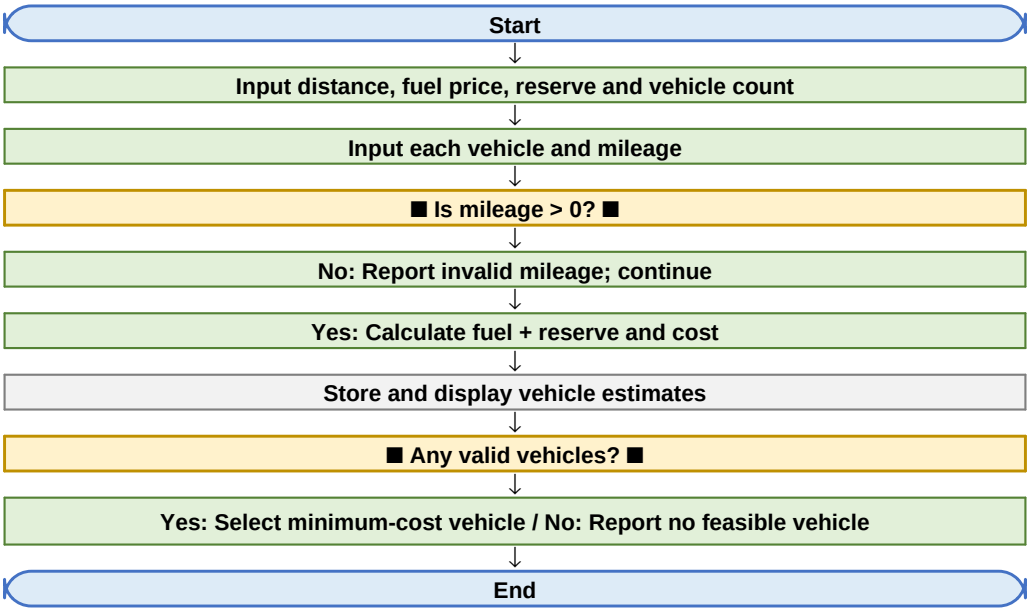
5. Calculate base fuel = distance ÷ mileage.

6. Add reserve: total fuel = base fuel + base fuel × reserve ÷ 100.

7. Calculate cost = total fuel × fuel price and store the result.

8. Print each vehicle’s fuel and cost, then report the lowest-cost vehicle if any valid vehicle exists.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

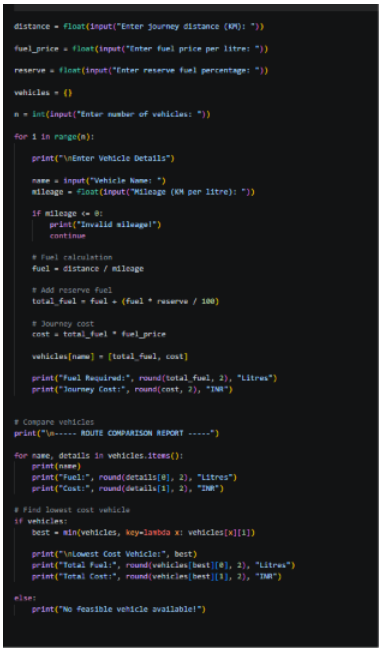
7. Pseudocode

```
BEGIN
```

```
READ distance, fuel_price, reserve, n
FOR each vehicle: READ name, mileage
IF mileage ≤ 0: DISPLAY invalid mileage; CONTINUE
fuel ← distance / mileage
total_fuel ← fuel + fuel × reserve / 100
cost ← total_fuel × fuel_price
STORE vehicle, total_fuel, cost
DISPLAY comparison report
IF vehicles exist: DISPLAY vehicle with minimum cost
ELSE DISPLAY no feasible vehicle
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for fuel consumption and route comparison.



```python
distance = float(input("Enter journey distance (KM): "))

fuel_price = float(input("Enter fuel price per litre: "))

reserve = float(input("Enter reserve fuel percentage: "))

vehicles = {}

n = int(input("Enter number of vehicles: "))

for i in range(n):

    print("\nEnter Vehicle Details")

    name = input("Vehicle Name: ")
    mileage = float(input("Mileage (KM per litre): "))

    if mileage <= 0:
        print("Invalid mileage!")
        continue

    # Fuel calculation
    fuel = distance / mileage

    # Add reserve fuel
    total_fuel = fuel + (fuel * reserve / 100)

    # Journey cost
    cost = total_fuel * fuel_price

    vehicles[name] = [total_fuel, cost]

    print("Fuel Required:", round(total_fuel, 2), "Litres")
    print("Journey Cost:", round(cost, 2), "INR")


# Compare vehicles
print("\n----- ROUTE COMPARISON REPORT -----")

for name, details in vehicles.items():
    print(name)
    print("Fuel:", round(details[0], 2), "Litres")
    print("Cost:", round(details[1], 2), "INR")

# Find lowest cost vehicle
if vehicles:
    best = min(vehicles, key=lambda x: vehicles[x][1])

    print("\nLowest Cost Vehicle:", best)
    print("Total Fuel:", round(vehicles[best][0], 2), "Litres")
    print("Total Cost:", round(vehicles[best][1], 2), "INR")

else:
    print("No feasible vehicle available!")
```

9. Conclusion

The system provides a simple cost comparison using mileage and reserve fuel. For real-world use, tank capacity, route-specific fuel prices, and a clearly defined feasibility rule should also be included.

Case Study 21: Traffic Violation Penalty Calculator

1. Make

Develop a Python program that calculates traffic penalties based on vehicle category, offence type, and repeat-offence status, and flags possible disqualification.

2. Problem Statement

Traffic penalty calculations must apply the correct fine rules consistently and identify cases that may require licence disqualification or other enforcement action.

3. Objective

Record violations, apply the relevant penalty schedule, calculate final fines, and flag disqualification conditions.

4. Assumptions

• Fine amounts and disqualification thresholds must be supplied by the relevant jurisdiction; they are not shown in the provided screenshot.

• Vehicle category, offence type, and repeat status are assumed to be entered from a verified list.

• This case's uploaded screenshot appears to show chemical inventory quantity and compatibility checks, not a traffic-violation calculator. It is included as supplied evidence, but it does not demonstrate the stated traffic-penalty logic.

5. Algorithm

1. Define an authoritative fine table by offence and vehicle category.

2. Read the number of violation records.

3. For each record, read vehicle category, offence type, and repeat-offence status.

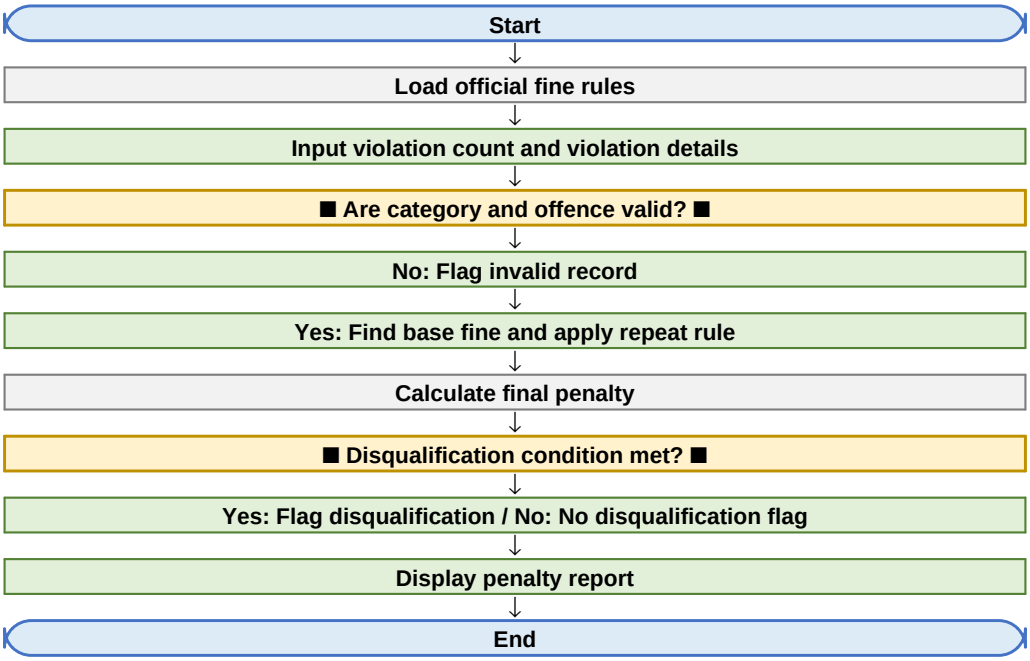
4. Validate the inputs against permitted categories and offences.

5. Look up the base fine and apply the repeat-offence rule.

6. Calculate the final penalty and check disqualification conditions.

7. Display each violation, fine, and any disqualification warning; total penalties if required.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

7. Pseudocode

```
BEGIN
LOAD fine_rules and disqualification_rules
READ n
FOR each violation: READ vehicle_category, offence, repeat_status
IF inputs are invalid: FLAG record; CONTINUE
base_fine ← fine_rules[vehicle_category, offence]
final_fine ← APPLY repeat-offence rule(base_fine, repeat_status)
disqualified ← CHECK disqualification_rules
DISPLAY final_fine and disqualified status
END FOR
END
```

8. Screenshot of Python Source Code

Screenshot 3 was supplied for this position; it depicts chemical inventory validation rather than traffic penalty calculation.



```python
chemicals = []

n = int(input("Enter number of chemicals: "))

for i in range(n):

    print("\nEnter Chemical Details")

    name = input("Enter chemical name: ")
    category = input("Enter storage category: ")
    quantity = float(input("Enter quantity (litres): "))

    max_quantity = float(input("Enter maximum allowed quantity: "))

    if quantity > max_quantity:
        print("Invalid: Maximum quantity exceeded")

    else:
        print("Valid quantity")

    chemicals.append([name, category, quantity])

# Check compatibility
print("\n--- INVENTORY REPORT ---")

for c in chemicals:
    print("\nChemical:", c[0])
    print("Category:", c[1])
    print("Quantity:", c[2])

# Example compatibility restriction
for i in range(len(chemicals)):
    for j in range(i + 1, len(chemicals)):

        if (chemicals[i][1] == "Acid" and
            chemicals[j][1] == "Base") or \
           (chemicals[i][1] == "Base" and
            chemicals[j][1] == "Acid"):

            print("Invalid combination:",
                  chemicals[i][0], "and", chemicals[j][0])
```

9. Conclusion

A rule-driven calculator can improve consistency and transparency in penalty processing. Fine tables and disqualification rules must be kept current and verified against local law. The supplied screenshot should be replaced with the actual traffic-violation program screenshot when available.

Case Study 30: Student Email and Identifier Validation

1. Make

Write a Python program to validate student email-like strings and registration identifiers using simple formatting rules.

2. Problem Statement

Student records may contain malformed email addresses or registration IDs. Early validation helps detect missing separators, incorrect prefixes, and invalid character groups.

3. Objective

Check email structure and registration-ID format and report each value as valid or invalid.

4. Assumptions

• The screenshot's email rule checks for '@' and a period '.' in the input.

• The registration ID rule requires the prefix 'STU', followed by digits in the expected positions, and a total length of eight characters, as expressed in the code.

• These are simplified classroom rules, not complete standards for all valid email addresses or institutional identifiers.

5. Algorithm

1. Read the student email and registration ID.

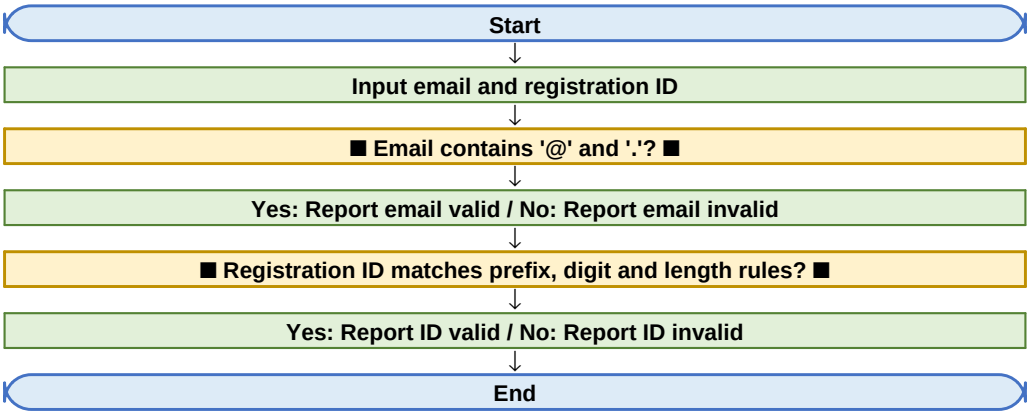
2. Check whether the email contains '@' and '.'.

3. Print Email is Valid if both characters are present; otherwise print Email is Invalid.

4. Check that the registration ID starts with 'STU', the selected substring contains digits, and the total length is eight.

5. Print Registration ID is Valid or Registration ID is Invalid.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

7. Pseudocode

```
BEGIN
READ&amp;nbsp;email,&amp;nbsp;reg_id
IF&amp;nbsp;'@'&amp;nbsp;IN&amp;nbsp;email&amp;nbsp;AND&amp;nbsp;'.'&amp;nbsp;IN&amp;nbsp;email:&amp;nbsp;DISPLAY&amp;nbsp;email&amp;nbsp;valid
ELSE:&amp;nbsp;DISPLAY&amp;nbsp;email&amp;nbsp;invalid
IF&amp;nbsp;reg_id&amp;nbsp;starts&amp;nbsp;with&amp;nbsp;'STU'&amp;nbsp;AND&amp;nbsp;required&amp;nbsp;characters&amp;nbsp;are&amp;nbsp;digits&amp;nbsp;AND&amp;nbsp;length&amp;nbsp;=&amp;nbsp;8:
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;DISPLAY&amp;nbsp;registration&amp;nbsp;ID&amp;nbsp;valid
ELSE:&amp;nbsp;DISPLAY&amp;nbsp;registration&amp;nbsp;ID&amp;nbsp;invalid
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for email and registration-ID validation.

```python
email = input("Enter student email: ")
reg_id = input("Enter registration ID: ")

# Email validation
if "@" in email and "." in email and " " not in email:
    print("Email is Valid")
else:
    print("Email is Invalid")

# Registration ID validation
if reg_id.startswith("STU") and reg_id[3:].isdigit() and len(reg_id) == 8:
    print("Registration ID is Valid")
else:
    print("Registration ID is Invalid")
```

9. Conclusion

The program demonstrates basic input validation and gives immediate feedback. Production systems should use more complete email checks and the institution’s exact registration-ID specification.

Case Study 37: Nested Loop Calendar Generator

1. Make

Generate a text calendar for a month using nested loops, the number of days, and the starting weekday.

2. Problem Statement

A calendar must align dates under the correct weekdays and wrap dates to a new line after Saturday, while handling different month lengths.

3. Objective

Print a readable monthly calendar with correct weekday alignment for valid inputs.

4. Assumptions

• The starting weekday is represented as a number from 0 to 6, where 0 is Sunday and 6 is Saturday, as suggested by the prompt.

• The number of days is assumed to be a valid month length (28–31).

• The supplied code uses nested loops to print leading blanks and then dates.

5. Algorithm

1. Read the number of days and starting weekday.

2. Print weekday headings from Sunday through Saturday.

3. Set the day counter to 1.

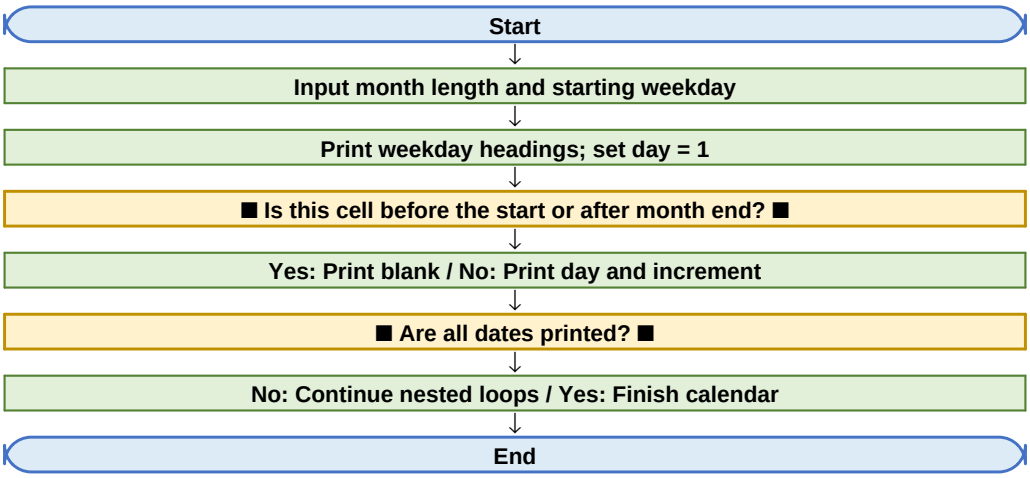
4. Use an outer loop for up to six calendar weeks and an inner loop for seven weekdays.

5. Print blanks before the first date according to the starting weekday.

6. Print each date and increment the counter until the month is complete.

7. Move to a new line after each week and stop when all dates are printed.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

7. Pseudocode

```
BEGIN
READ&amp;nbsp;days,&amp;nbsp;start
PRINT&amp;nbsp;weekday&amp;nbsp;headings
day&amp;nbsp;←&amp;nbsp;1
FOR&amp;nbsp;week&amp;nbsp;in&amp;nbsp;0..5:
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;FOR&amp;nbsp;weekday&amp;nbsp;in&amp;nbsp;0..6:
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;IF&amp;nbsp;first&amp;nbsp;week&amp;nbsp;and&amp;nbsp;weekday&amp;nbsp;
<&amp;nbsp;start:&amp;nbsp;PRINT&amp;nbsp;blank
```

        ELSE IF day ≤ days: PRINT day; day ← day + 1

        ELSE: PRINT blank

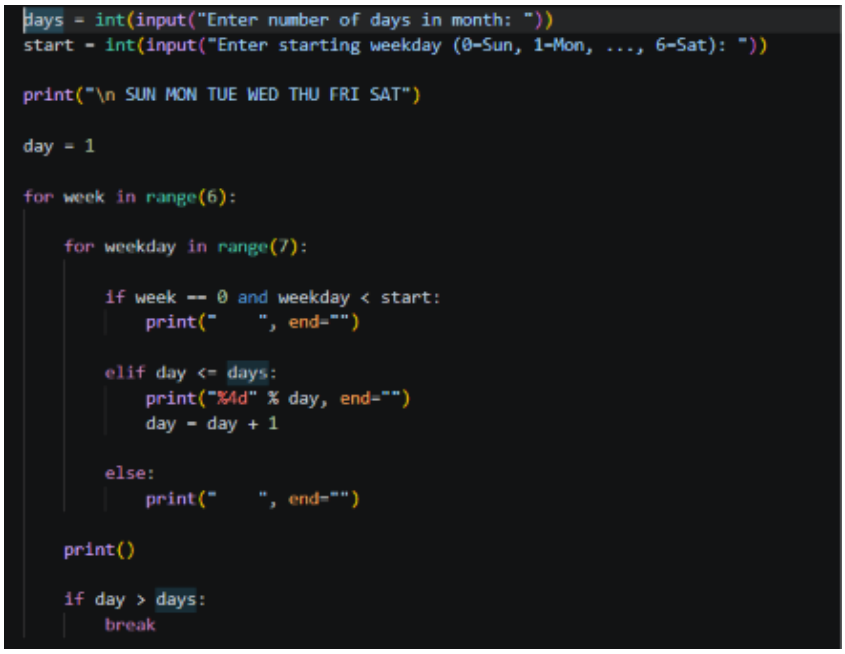
    PRINT newline

    IF day > days: BREAK

END

8. Screenshot of Python Source Code

Source-code screenshot supplied for the nested-loop calendar generator.



```
days = int(input("Enter number of days in month: "))
start = int(input("Enter starting weekday (0-Sun, 1-Mon, ..., 6-Sat): "))

print("\n SUN MON TUE WED THU FRI SAT")

day = 1

for week in range(6):

    for weekday in range(7):

        if week == 0 and weekday < start:
            print("    ", end="")

        elif day <= days:
            print("%4d" % day, end="")
            day = day + 1

        else:
            print("    ", end="")

    print()

    if day > days:
        break
```

9. Conclusion

Nested loops provide a straightforward way to format a monthly calendar. Validating the starting weekday and month length prevents incorrect alignment and unsupported dates.

Case Study 41: Nested List Classroom Occupancy Analysis

1. Make

Represent classrooms and their seats as nested lists, allocate students to available seats, preserve reserved positions, and count vacancies.

2. Problem Statement

Classroom seating data must distinguish reserved, empty, and occupied seats while avoiding duplicate allocations and accurately calculating available capacity.

3. Objective

Allocate students where possible, display classroom layouts, detect duplicate assignments, and report total, occupied, and vacant seats.

4. Assumptions

• Each classroom is represented as a list of seat labels.

• The values 'Empty' and 'Reserved' have special meanings.

• The code allocates each student to the first available seat marked 'Empty'.

• Seat and student data are assumed to follow the same naming conventions throughout.

5. Algorithm

1. Create a nested list representing the classrooms and seats.

2. Read or define the list of students.

3. For each student, search classrooms and seats for the first 'Empty' position.

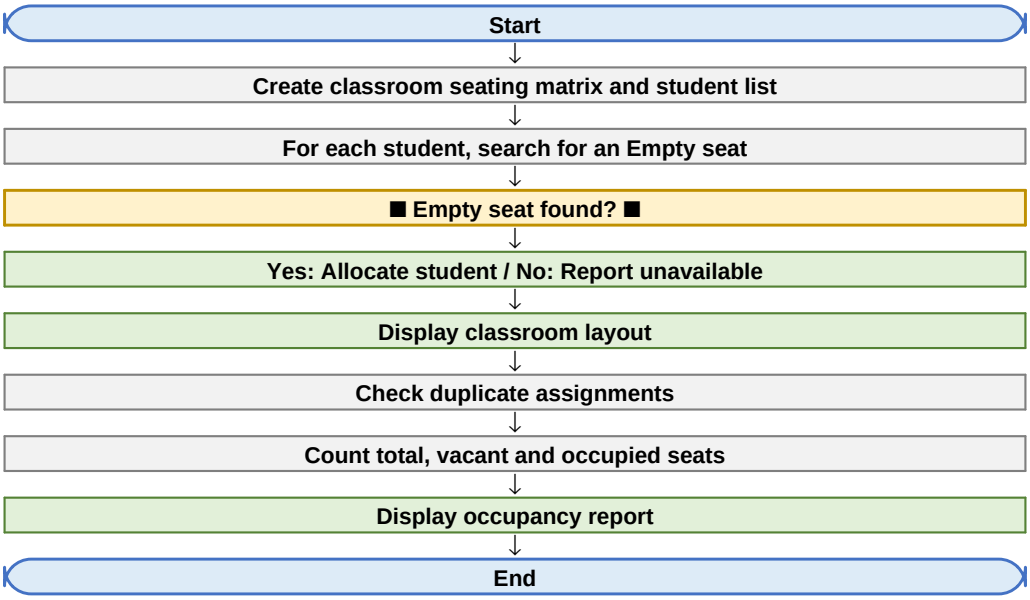
4. Assign the student to that position, or report no seat available.

5. Display classroom details.

6. Scan seats for duplicate non-empty assignments, ignoring 'Empty' and 'Reserved'.

7. Count total seats, vacant seats, and occupied seats; print the summary.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

7. Pseudocode

```
BEGIN
INITIALIZE classrooms and students
```

FOR each student:
    found ← FALSE
    FOR each classroom and seat:
        IF seat = 'Empty': assign student; found ← TRUE; BREAK
    IF NOT found: DISPLAY no seat available
DISPLAY classroom details
SCAN occupied seats and detect duplicates
COUNT total, vacant and occupied seats
DISPLAY counts
END

8. Screenshot of Python Source Code

Source-code screenshot supplied for classroom occupancy and seat allocation.



9. Conclusion

Nested lists make it possible to model multiple classrooms and their seating positions. The program helps with allocation and capacity reporting; duplicate checks should ignore reserved and empty markers.

Case Study 52: Hackathon Team Formation and Ranking

1. Make

Collect team details and round scores, validate team sizes, compute total scores, and rank teams using deterministic tie-breakers.

2. Problem Statement

Hackathon results need a consistent ranking method that combines multiple rounds while ensuring each team satisfies the permitted member count.

3. Objective

Validate team composition, calculate round-wise totals, rank valid teams, and display final scores in order.

4. Assumptions

• The supplied code treats team size below 2 or above 4 as invalid.

• Each team has three round scores.

• Total score is the sum of all three rounds.

• Teams are sorted by total score and final-round score in descending order, matching the code.

5. Algorithm

1. Read the number of teams.

2. For each team, read its name and member count.

3. If the team size is outside 2–4, report Invalid Team Size and skip score entry.

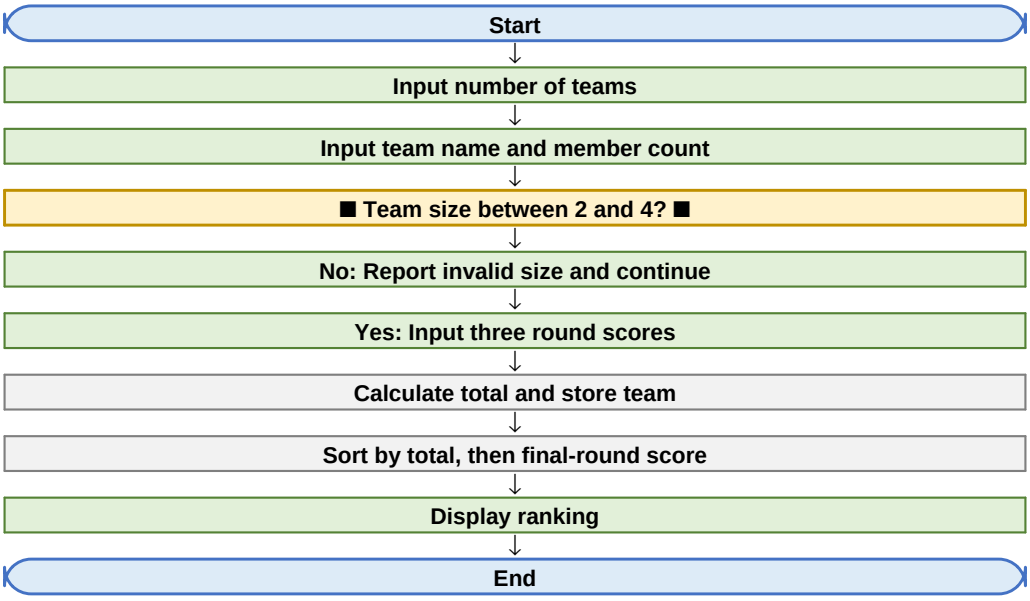
4. Read scores for rounds 1, 2, and 3.

5. Calculate the total score and store the team record.

6. Sort valid teams by total score, then round-three score, descending.

7. Print the hackathon ranking.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

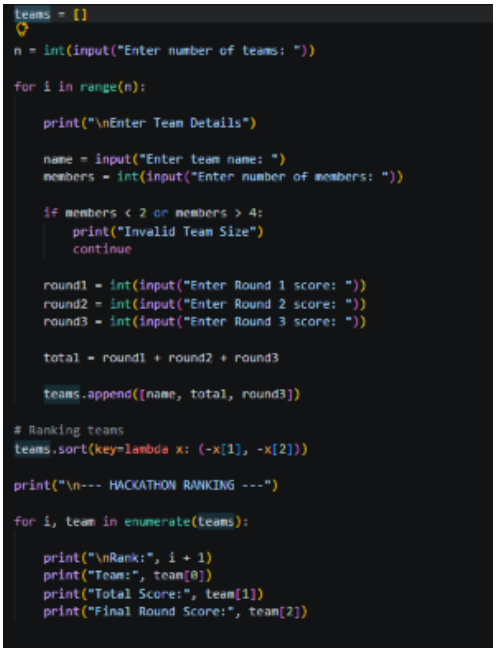
7. Pseudocode

```
BEGIN
READ&amp;nbsp;n
FOR&amp;nbsp;each&amp;nbsp;team:&amp;nbsp;READ&amp;nbsp;name,&amp;nbsp;members
```

```
IF members < 2 OR members > 4: DISPLAY invalid size; CONTINUE
READ round1, round2, round3
total ← round1 + round2 + round3
STORE team, total, round3
SORT teams by total DESC, round3 DESC
DISPLAY ranked teams
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for hackathon team score aggregation and ranking.



```python
teams = []

n = int(input("Enter number of teams: "))

for i in range(n):

    print("\nEnter Team Details")

    name = input("Enter team name: ")
    members = int(input("Enter number of members: "))

    if members < 2 or members > 4:
        print("Invalid Team Size")
        continue

    round1 = int(input("Enter Round 1 score: "))
    round2 = int(input("Enter Round 2 score: "))
    round3 = int(input("Enter Round 3 score: "))

    total = round1 + round2 + round3

    teams.append([name, total, round3])

# Ranking teams
teams.sort(key=lambda x: (-x[1], -x[2]))

print("\n--- HACKATHON RANKING ---")

for i, team in enumerate(teams):

    print("\nRank:", i + 1)
    print("Team:", team[0])
    print("Total Score:", team[1])
    print("Final Round Score:", team[2])
```

9. Conclusion

The program supports fair and repeatable ranking by using score totals and a final-round tie-breaker. A complete system should also define what happens when teams tie on every ranking key.

Case Study 61: Nested List Examination Seating Validation

1. Make

Validate an examination seating matrix for reserved seats, empty seats, invalid roll numbers, and duplicate student allocations.

2. Problem Statement

Examination seating plans must ensure each roll number is valid and assigned no more than once, while reserved and empty seats are identified correctly.

3. Objective

Inspect every seat, classify its status, detect invalid or duplicate roll numbers, and summarize seating capacity.

4. Assumptions

• 'R' indicates a reserved seat and None indicates an empty seat.

• The permitted roll-number list is supplied separately.

• A roll number may appear only once in the seating matrix.

• The screenshot’s checks are performed in sequence: reserved, empty, invalid, duplicate, then valid.

5. Algorithm

1. Define the seating matrix and valid roll-number list.

2. Create an empty list to track assigned students.

3. Visit every seat in every row.

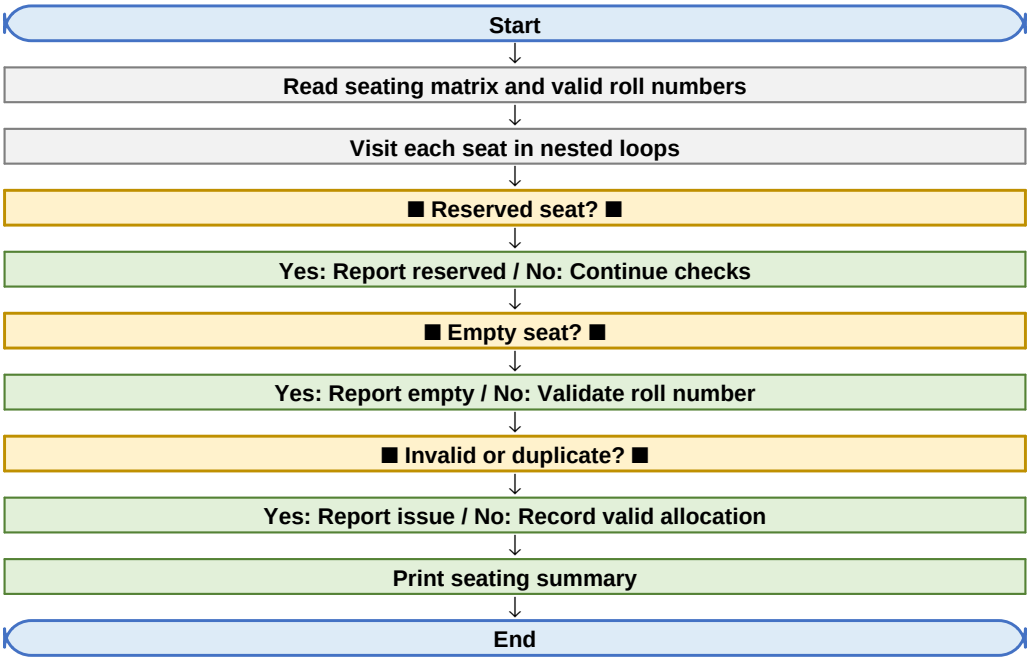
4. If the seat is 'R', report reserved; if None, report empty.

5. If the roll number is not in the valid list, report invalid allocation.

6. If it is already assigned, report duplicate; otherwise store it and report valid.

7. Print total seats, allocated students, and empty seats.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

7. Pseudocode

```
BEGIN
```

INITIALIZE assigned ← empty list

FOR each seat in matrix:

    IF seat = 'R': REPORT reserved

    ELSE IF seat = None: REPORT empty

    ELSE IF seat NOT IN valid_roll_numbers: REPORT invalid allocation

    ELSE IF seat IN assigned: REPORT duplicate roll number

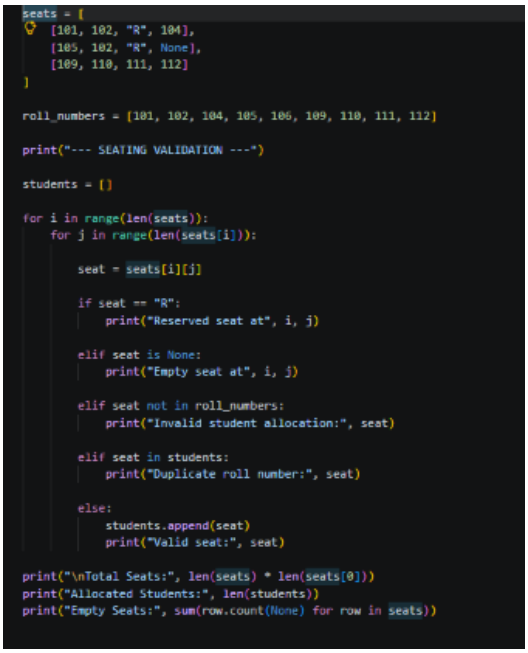
    ELSE: APPEND seat to assigned; REPORT valid seat

DISPLAY total seats, allocated count, empty count

END

8. Screenshot of Python Source Code

Source-code screenshot supplied for nested-list examination seating validation.



```
seats = [
    [101, 102, "R", 104],
    [105, 102, "R", None],
    [109, 110, 111, 112]
]

roll_numbers = [101, 102, 104, 105, 106, 109, 110, 111, 112]

print("--- SEATING VALIDATION ---")

students = []

for i in range(len(seats)):
    for j in range(len(seats[i])):

        seat = seats[i][j]

        if seat == "R":
            print("Reserved seat at", i, j)

        elif seat is None:
            print("Empty seat at", i, j)

        elif seat not in roll_numbers:
            print("Invalid student allocation:", seat)

        elif seat in students:
            print("Duplicate roll number:", seat)

        else:
            students.append(seat)
            print("Valid seat:", seat)

print("\nTotal Seats:", len(seats) * len(seats[0]))
print("Allocated Students:", len(students))
print("Empty Seats:", sum(row.count(None) for row in seats))
```

9. Conclusion

The validation process improves examination seating accuracy by detecting reserved positions, vacancies, invalid roll numbers, and duplicate assignments. Summary counts help staff verify room usage.

Case Study 69: Nested List Sports Performance Analysis

1. Make

Analyse player scores across three matches, calculate summary measures, identify trends, and rank players using defined tie-breakers.

2. Problem Statement

Coaches need a compact comparison of players across matches, including total and average scores, best and worst performance, score trend, and consistency range.

3. Objective

Compute performance statistics for each player and produce a ranked sports performance report.

4. Assumptions

• Each player provides three numeric match scores.

• Total is the sum of the scores and average is total divided by three.

• Highest and lowest values are taken from the three matches.

• Players are ranked by total score and the first-match score, descending, as implemented in the screenshot.

• Trend is determined by comparing consecutive match scores.

5. Algorithm

1. Read the number of players.

2. For each player, read the name and three match scores.

3. Calculate total, average, highest score, and lowest score.

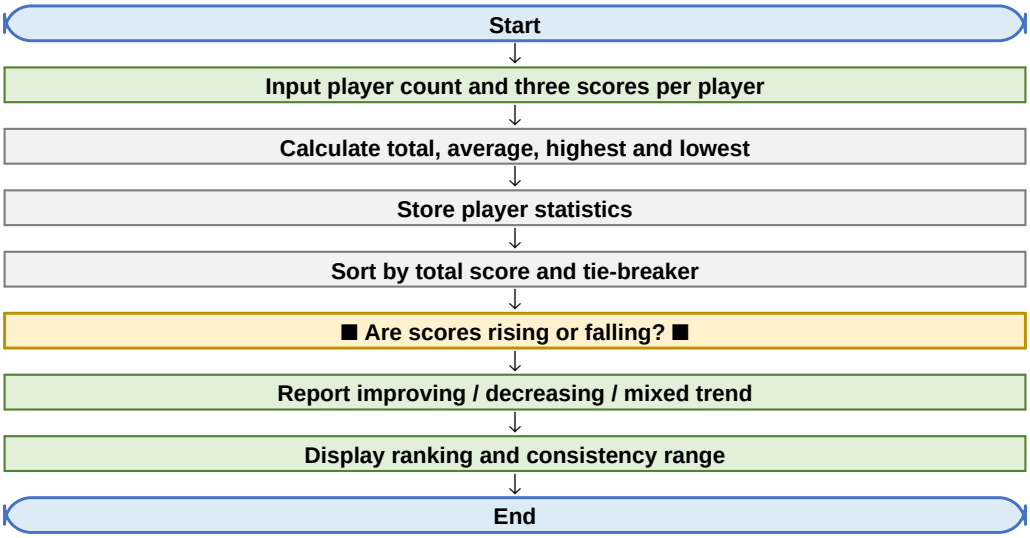
4. Store each player's record.

5. Sort players by total score and first-match score in descending order.

6. For each ranked player, display statistics and trend: improving, decreasing, or mixed.

7. Display the consistency range as highest minus lowest.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

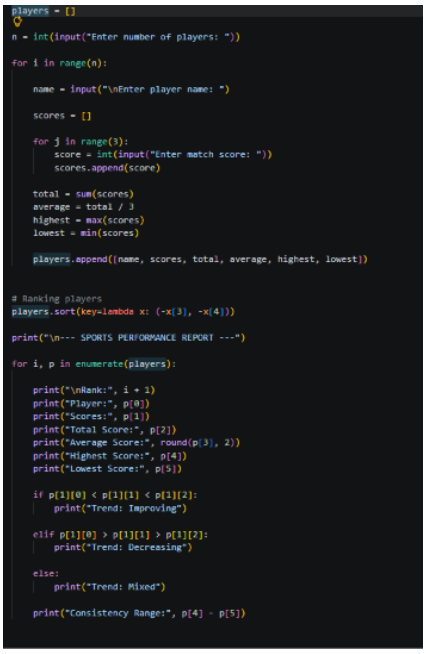
7. Pseudocode

```
BEGIN
READ&amp;nbsp;n
FOR&amp;nbsp;each&amp;nbsp;player:&amp;nbsp;READ&amp;nbsp;name&amp;nbsp;and&amp;nbsp;three&amp;nbsp;scores
```

```
total ← SUM(scores)
average ← total / 3
highest ← MAX(scores); lowest ← MIN(scores)
STORE player statistics
SORT by total DESC, first-match score DESC
FOR each ranked player:
    DETERMINE trend from consecutive scores
    DISPLAY rank, scores, total, average, high, l
ow, trend, range
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for sports performance analysis.



```python
players = []

n = int(input("Enter number of players: "))

for i in range(n):

    name = input("\nEnter player name: ")

    scores = []

    for j in range(3):
        score = int(input("Enter match score: "))
        scores.append(score)

    total = sum(scores)
    average = total / 3
    highest = max(scores)
    lowest = min(scores)

    players.append((name, scores, total, average, highest, lowest))


# Ranking players
players.sort(key=lambda x: (-x[3], -x[4]))

print("\n--- SPORTS PERFORMANCE REPORT ---")

for i, p in enumerate(players):

    print("\nRank:", i + 1)
    print("Player:", p[0])
    print("Scores:", p[1])
    print("Total Score:", p[2])
    print("Average Score:", round(p[3], 2))
    print("Highest Score:", p[4])
    print("Lowest Score:", p[5])

    if p[1][0] < p[1][1] < p[1][2]:
        print("Trend: Improving")

    elif p[1][0] > p[1][1] > p[1][2]:
        print("Trend: Decreasing")

    else:
        print("Trend: Mixed")

    print("Consistency Range:", p[4] - p[5])
```

9. Conclusion

The program turns match scores into comparable statistics and ranking information. Trend and range metrics help distinguish consistent performance from fluctuating results.

Case Study 82: Multi-Criteria Internship Candidate Ranking

1. Make

Evaluate internship candidates using marks, assessment score, and attendance; retain eligible candidates and rank them by predefined criteria.

2. Problem Statement

Internship selection requires transparent eligibility rules and a repeatable ranking process when many candidates apply.

3. Objective

Apply minimum thresholds, calculate a combined score, and display a ranked list of qualified candidates.

4. Assumptions

• The supplied code requires marks ≥ 60, assessment score ≥ 50, and attendance ≥ 75.

• Total score is marks plus assessment score.

• Eligible candidates are sorted by total score, marks, and assessment score in descending order, matching the code.

• Only eligible candidates appear in the ranking list.

5. Algorithm

1. Read the number of candidates.

2. For each candidate, read name, marks, assessment score, and attendance.

3. Check all eligibility thresholds.

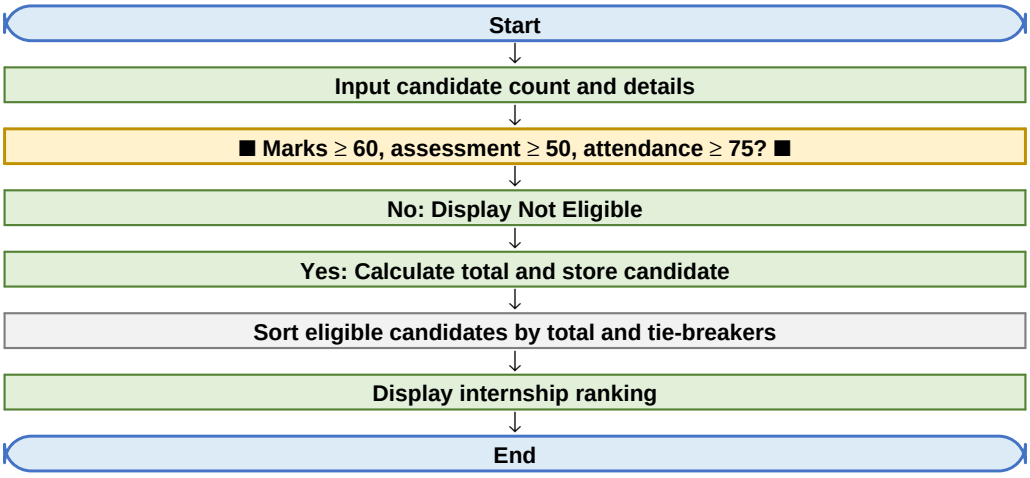
4. If eligible, calculate total = marks + assessment score and store the candidate record.

5. Otherwise, report Not Eligible.

6. Sort eligible candidates by total, marks, and assessment score in descending order.

7. Print the internship ranking with each candidate’s details.

6. Flowchart



Flowchart symbol key: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision; arrows = direction of control.

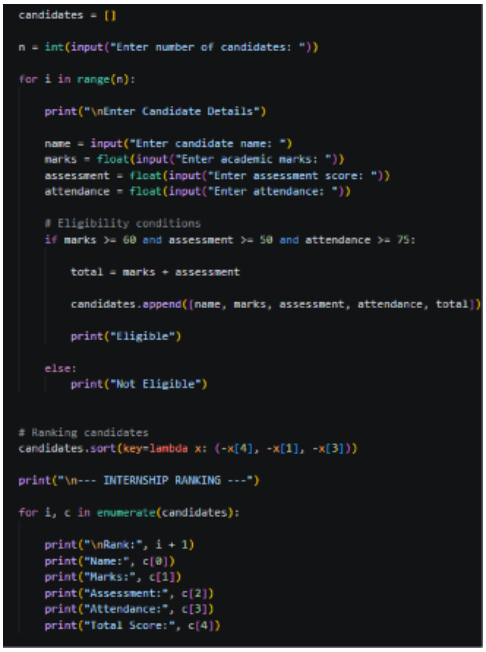
7. Pseudocode

```
BEGIN
READ&amp;nbsp;n
FOR&amp;nbsp;each&amp;nbsp;candidate:&amp;nbsp;READ&amp;nbsp;name,&amp;nbsp;marks,&amp;nbsp;assessment,&amp;nbsp;attendance
IF&amp;nbsp;marks&amp;nbsp;≥&amp;nbsp;60&amp;nbsp;AND&amp;nbsp;assessment&amp;nbsp;≥&amp;nbsp;50&amp;nbsp;AND&amp;nbsp;attendance&amp;nbsp;≥
&amp;nbsp;75:
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;total&amp;nbsp;←&amp;nbsp;marks&amp;nbsp;+&amp;nbsp;assessment
&amp;nbsp;&amp;nbsp;&amp;nbsp;&amp;nbsp;APPEND&amp;nbsp;candidate&amp;nbsp;details&amp;nbsp;and&amp;nbsp;total
ELSE:&amp;nbsp;DISPLAY&amp;nbsp;Not&amp;nbsp;Eligible
```

```
SORT candidates by total DESC, marks DESC, assessment DESC
DISPLAY internship ranking
END
```

8. Screenshot of Python Source Code

Source-code screenshot supplied for internship eligibility and ranking.



```python
candidates = []

n = int(input("Enter number of candidates: "))

for i in range(n):

    print("\nEnter Candidate Details")

    name = input("Enter candidate name: ")
    marks = float(input("Enter academic marks: "))
    assessment = float(input("Enter assessment score: "))
    attendance = float(input("Enter attendance: "))

    # Eligibility conditions
    if marks >= 60 and assessment >= 50 and attendance >= 75:

        total = marks + assessment

        candidates.append([name, marks, assessment, attendance, total])

        print("Eligible")

    else:
        print("Not Eligible")


# Ranking candidates
candidates.sort(key=lambda x: (-x[4], -x[1], -x[3]))

print("\n--- INTERNSHIP RANKING ---")

for i, c in enumerate(candidates):

    print("\nRank:", i + 1)
    print("Name:", c[0])
    print("Marks:", c[1])
    print("Assessment:", c[2])
    print("Attendance:", c[3])
    print("Total Score:", c[4])
```

9. Conclusion

The system applies clear thresholds and ranks eligible applicants consistently. The selection committee should document tie-breakers and verify that the scoring formula reflects the official internship policy.